

Sessão 2.1: Estruturas de Dados em R

Norberto Jorge Gonçalves

6 de Julho de 2025

Contents

1. Estruturas de Dados em R	1
1.1. Vetores (Vectors)	1
1.2. Matrizes (Matrices)	2
1.3. Arrays (Arrays)	3
1.4. Listas (Lists)	3
1.5. Data Frames (Data Frames) - O Nosso Foco!	4

1. Estruturas de Dados em R

Nesta sessão, vamos aprofundar os diferentes tipos de estruturas de dados que o R utiliza para organizar e armazenar informação. Compreender estas estruturas é fundamental para manipular e analisar dados eficazmente.

1.1. Vetores (Vectors)

Um vetor é a estrutura de dados mais básica em R. É uma sequência de elementos do **mesmo tipo**.

- **Númericos:** Números inteiros ou decimais.
- **Caracteres (Strings):** Texto.
- **Lógicos (Booleans):** TRUE ou FALSE.

Podemos criar vetores usando a função `c()` (de “combine” ou “concatenate”).

```
# Vetor numérico
idades <- c(18, 19, 17, 20, 18)
print(idades)

## [1] 18 19 17 20 18

class(idades) # Para verificar o tipo

## [1] "numeric"

# Vetor de caracteres
nomes <- c("João", "Maria", "Pedro", "Ana")
print(nomes)

## [1] "João" "Maria" "Pedro" "Ana"

class(nomes)

## [1] "character"
```

```
# Vetor lógico
aprovado <- c(TRUE, FALSE, TRUE, TRUE)
print(aprovado)
```

```
## [1] TRUE FALSE TRUE TRUE
```

```
class(aprovado)
```

```
## [1] "logical"
```

```
# Aceder a elementos de um vetor (usando [])
print(idades[1]) # Primeiro elemento
```

```
## [1] 18
```

```
print(nomes[c(2, 4)]) # Segundo e quarto elementos
```

```
## [1] "Maria" "Ana"
```

Importante: Todos os elementos de um vetor devem ser do mesmo tipo. Se tentarmos misturar, o R fará uma “coerção” para o tipo mais flexível (ex: números tornam-se caracteres se misturados com texto).

```
# Exemplo de coerção
vetor_misto <- c(1, "dois", 3)
print(vetor_misto)
```

```
## [1] "1" "dois" "3"
```

```
class(vetor_misto) # O que aconteceu?
```

```
## [1] "character"
```

1.2. Matrizes (Matrices)

Uma matriz é uma coleção bidimensional de elementos do mesmo tipo, organizada em linhas e colunas. Pensem numa tabela onde todas as células contêm números, ou todas contêm texto. Podemos criar matrizes com a função ‘matrix()’.

```
# Criar uma matriz de 2 linhas e 3 colunas
matriz_numeros <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3)
print(matriz_numeros)
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
# Preencher por linha (por defeito é por coluna)
matriz_por_linha <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, byrow = TRUE)
print(matriz_por_linha)
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
```

```
# Aceder a elementos de uma matriz [linha, coluna]
print(matriz_numeros[1, 2]) # Elemento da 1ª linha, 2ª coluna
```

```
## [1] 3
```

```
print(matriz_numeros[, 1]) # Toda a 1ª coluna
```

```
## [1] 1 2
print(matriz_numeros[2, ]) # Toda a 2ª linha

## [1] 2 4 6
```

1.3. Arrays (Arrays)

Arrays são extensões das matrizes para mais de duas dimensões. Raramente usados em análises de dados introdutórias, mas útil saber que existem.

```
# Exemplo de um array 2x3x2
array_exemplo <- array(1:12, dim = c(2, 3, 2))
print(array_exemplo)
```

```
## , , 1
##
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
##
## , , 2
##
##      [,1] [,2] [,3]
## [1,]    7    9   11
## [2,]    8   10   12
```

1.4. Listas (Lists)

Uma lista é uma coleção de objetos que podem ser de **diferentes tipos** e tamanhos. Pensem numa lista como uma “caixa” que pode conter outras “caixas” (vetores, matrizes, data frames, ou até outras listas). Podemos criar listas com a função `list()`.

```
# Criar uma lista com diferentes tipos de objetos
minha_lista <- list(
  nome = "Carlos",
  idade = 17,
  notas = c(15, 18, 14),
  disciplinas = c("Matemática", "Física"),
  informacao_extra = matrix(c(1,2,3,4), nrow = 2)
)
print(minha_lista)
```

```
## $nome
## [1] "Carlos"
##
## $idade
## [1] 17
##
## $notas
## [1] 15 18 14
##
## $disciplinas
## [1] "Matemática" "Física"
##
## $informacao_extra
##      [,1] [,2]
```

```
## [1,] 1 3
## [2,] 2 4

# Aceder a elementos de uma lista
# Usar `$` para aceder pelo nome
print(minha_lista$nome)

## [1] "Carlos"

print(minha_lista$notas)

## [1] 15 18 14

# Usar `[[ ]]` para aceder pelo índice (e manter o tipo do elemento)
print(minha_lista[[3]])

## [1] 15 18 14

class(minha_lista[[3]])

## [1] "numeric"

# Usar `[ ]` para aceder pelo índice (e obter uma sub-lista)
print(minha_lista[3])

## $notas
## [1] 15 18 14

class(minha_lista[3]) # Note que o resultado ainda é uma lista!

## [1] "list"
```

1.5. Data Frames (Data Frames) - O Nosso Foco!

Um **data frame** é a estrutura de dados **mais importante** para a Ciência de Dados em R. É uma lista de vetores de igual comprimento, onde cada vetor representa uma coluna e cada elemento do vetor representa uma linha. Pensem num **data frame** como uma **tabela** ou uma **folha de cálculo**, onde: * Cada **coluna** pode ter um tipo de dados diferente (numérico, texto, lógico, etc.). * Cada **linha** representa uma observação ou um registo. Podemos criar um **data frame** usando a função `data.frame()`.

```
# Criar um data frame
alunos_df <- data.frame(
  Nome = c("Ana", "Bruno", "Carla", "Diogo"),
  Idade = c(17, 18, 17, 19),
  Altura_cm = c(165, 180, 170, 175),
  Genero = c("F", "M", "F", "M"),
  Nota_Mat = c(16, 14, 18, 15)
)
print(alunos_df)
```

```
##      Nome Idade Altura_cm Genero Nota_Mat
## 1   Ana    17      165      F      16
## 2 Bruno    18      180      M      14
## 3 Carla    17      170      F      18
## 4 Diogo    19      175      M      15
```

```
# Ver a estrutura do data frame
str(alunos_df)
```

```
## 'data.frame': 4 obs. of 5 variables:
```

```
## $ Nome      : chr  "Ana" "Bruno" "Carla" "Diogo"
## $ Idade     : num   17 18 17 19
## $ Altura_cm: num   165 180 170 175
## $ Genero    : chr   "F" "M" "F" "M"
## $ Nota_Mat : num    16 14 18 15

# Ver as dimensões (linhas x colunas)
dim(alunos_df)

## [1] 4 5

# Ver os nomes das colunas
colnames(alunos_df)

## [1] "Nome"      "Idade"      "Altura_cm" "Genero"     "Nota_Mat"

# Aceder a colunas (variáveis) de um data frame
print(alunos_df$Nome)

## [1] "Ana"      "Bruno" "Carla" "Diogo"

print(alunos_df[["Idade"]])

## [1] 17 18 17 19

# Aceder a linhas e colunas [linha, coluna]
print(alunos_df[1, ]) # Primeira linha

##   Nome Idade Altura_cm Genero Nota_Mat
## 1  Ana   17      165      F      16

print(alunos_df[, "Altura_cm"]) # Coluna 'Altura_cm'

## [1] 165 180 170 175

print(alunos_df[2, 3]) # Elemento da 2ª linha, 3ª coluna

## [1] 180
```

Os data frames são incrivelmente versáteis e serão a base de quase todas as nossas análises.